Kumbhakarna - The Sleepless Curse

PINAKACTF 2026 | Category: Reverse Engineering | Flag format: PINAKA{...}

Flag: PINAKA{kumbhakarna_wakes_when_y0u_model_the_machine_not_brute_it}

Summary

kumbha is an interpreter for a custom 24-bit virtual machine ("KUMBHA-24") and rom.bin is its program. The ROM mixes the four 24-bit invocation words through a chain of ones'-complement addition, GF(2^24) multiplication (against constants it decrypts at runtime) and an LCG-XOR, then writes the accumulator R7 to a verdict port. If that value is 0, the machine prints AWAKE. Every stage is individually invertible, so the correct invocation is computed, not brute-forced (96 bits).

Step 1 - Recon

file shows kumbha is a stripped PIE x86-64 ELF and rom.bin is raw data (780 bytes). CHALLENGE.txt frames it as reverse engineering: a machine that speaks an unknown language. A machine plus an opaque data blob points to a bytecode VM: the ELF is the interpreter and rom.bin is the program.

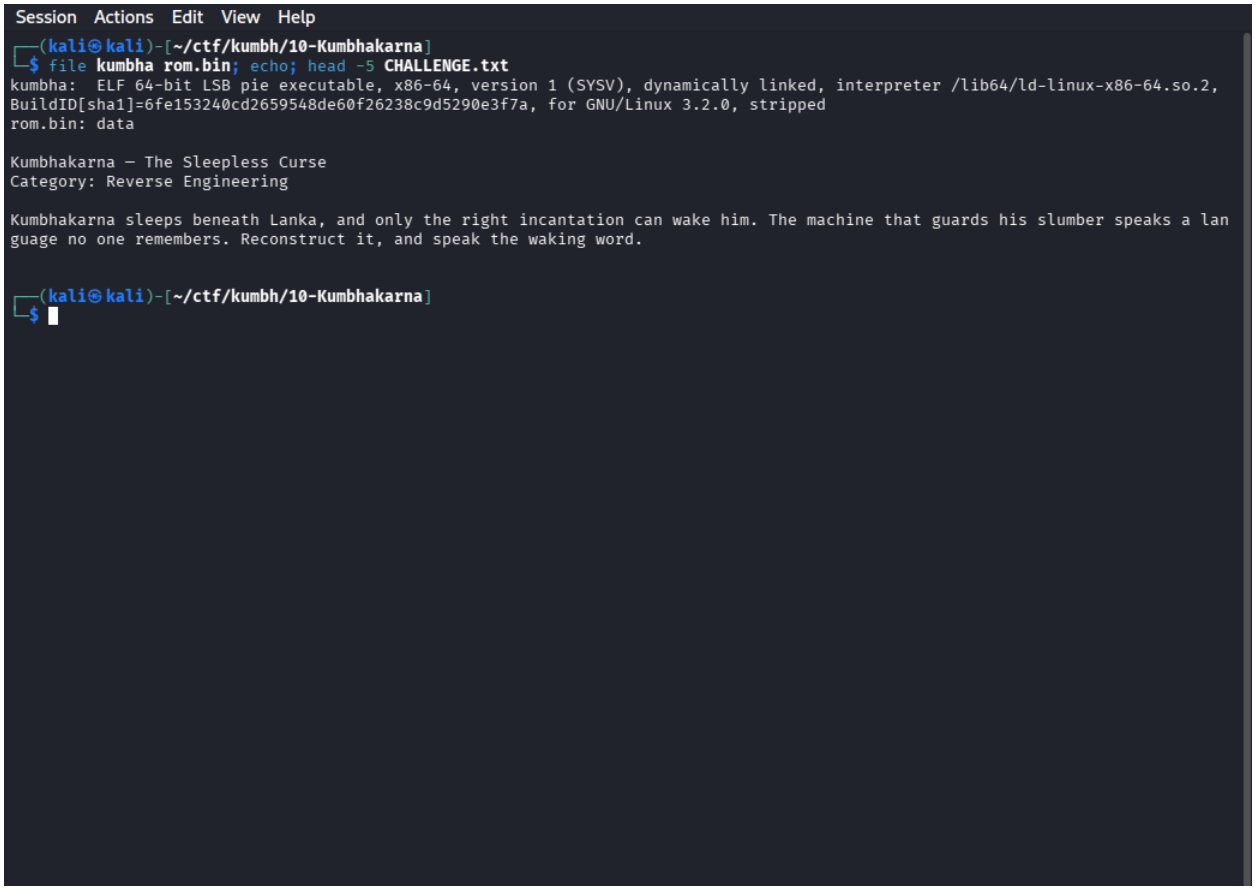


Fig 1 - file(1) output and the challenge prompt

Step 2 - Reverse the interpreter and the ROM

Disassembling main reveals a ~34-opcode switch. Key findings:

```
- Registers and instruction words are 24-bit. rom.bin stores 260 instructions as
  24-bit big-endian words; the binary zero-extends each to 32-bit LE.
- Verdict port 0xf50: OUT 0xf50,Rx sets an AWAKE flag + (Rx & 0xffffff);
  AWAKE is taken only when that value == 0.
- Input: IN port 0xf40 returns the next invocation word per read.
```

```
- Decrypt engine (ports 0xf10-0xf15): XOR-decrypts 4 code words at 0x100 with an
  LCG keystream seeded 0xc0de -> target constants T0..T3 in data memory.
- ADD is ones'-complement (mod 2^24-1); GMUL is GF(2^24), poly 0x100001b;
   PRNG is an LCG x*0x19660d+0x3c6ef3, init 0x9a1337, stepped once per stage (port
0xf31).
```

A 24-bit decoder (disasm.py) reads the ROM: engine setup, the 4x unrolled check loop, and the final OUT 0xf50 / HLT.

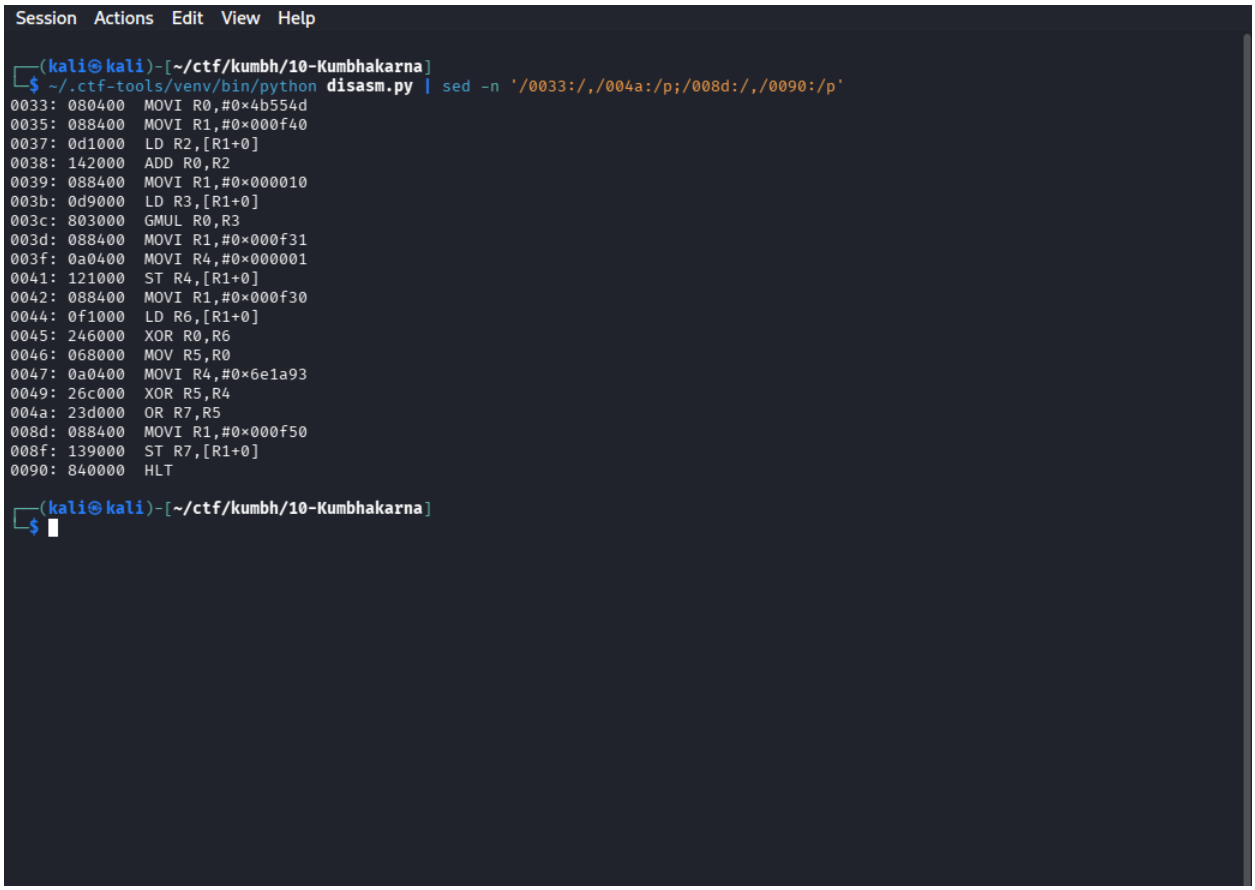


Fig 2 - KUMBHA-24 disassembly of the check routine

The check chains R0 (starting at 0x4b554d = "KUM") across all four words:

```
for k in 0..3:
    R0 = ocadd(R0, input[k])     # ones'-complement add
    R0 = gmul(R0, T[k])          # GF(2^24) multiply by decrypted target
    prng = lcg(prng); R0 ^= prng # advance + XOR keystream
    R7 |= (R0 ^ const[k])        # const=[6e1a93,69ad85,387eda,a30740]
```

R7 (OR of the four residues) goes to port 0xf50. For R7 == 0, every (R0 ^ const[k]) must be 0, i.e. after stage k the accumulator must equal const[k].

Step 3 - Invert the chain

Since R0 after each stage is forced to const[k], each stage inverts independently: undo the PRNG XOR, multiply by the GF inverse of T[k], then undo the ones'-complement add to recover input[k]. solve.py decrypts T0..T3, builds the GF(2^24) field + inverse, and walks the chain backwards.



Fig 3 - solve.py recovers the invocation words

Recovered invocation: 574B31 4E6752 41533A 6F6D21. The decrypted target T3 = a5b4c3 also matched R3 in the provided sample traces - a useful sanity check.

Step 4 - Verify on the binary (oracle)

Running the real interpreter with the recovered words prints AWAKE and the final trace tag d1ddfd42 - confirming the solve before submitting.

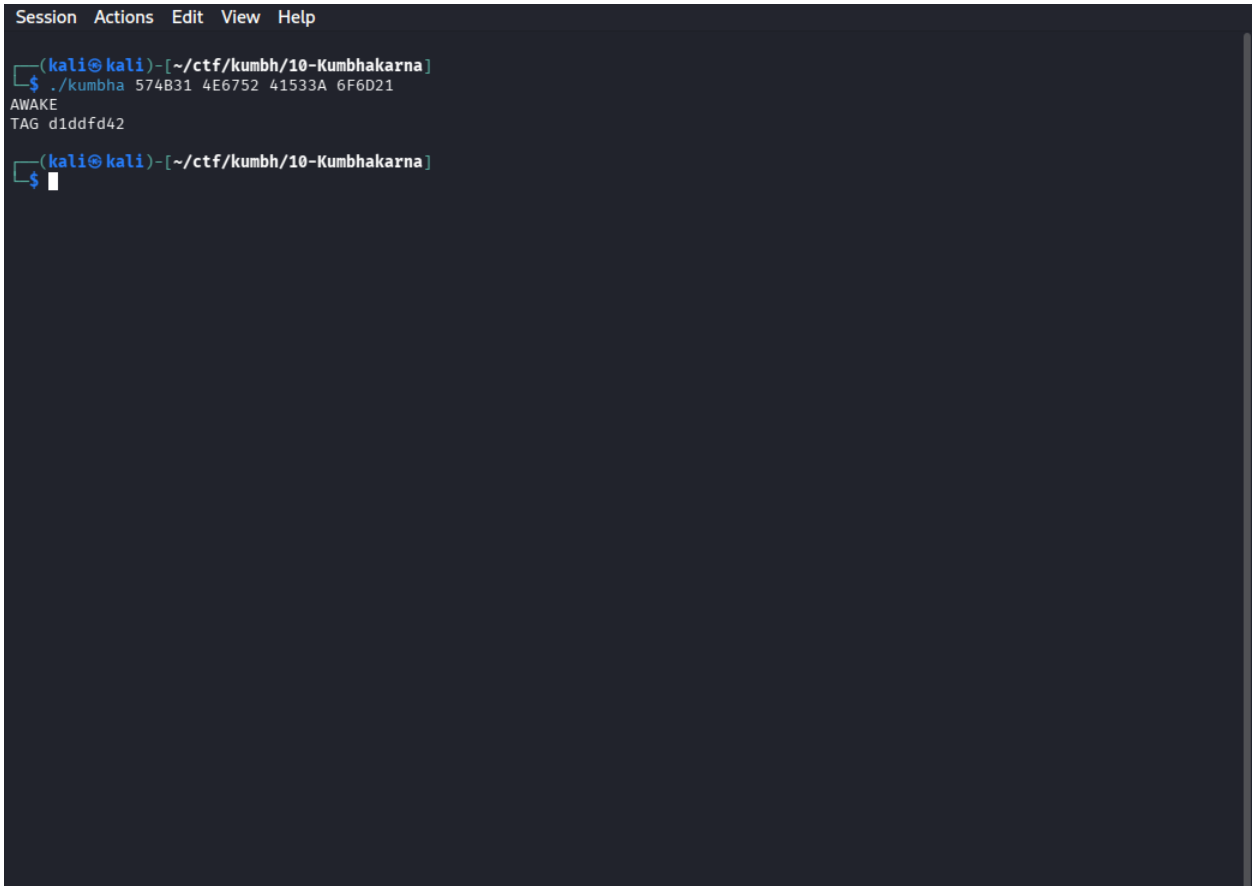


Fig 4 - the binary prints AWAKE + TAG d1ddfd42

Step 5 - Retrieve the flag

The validator expects the four invocation words plus the AWAKE trace tag at POST /verify. It returns the flag.

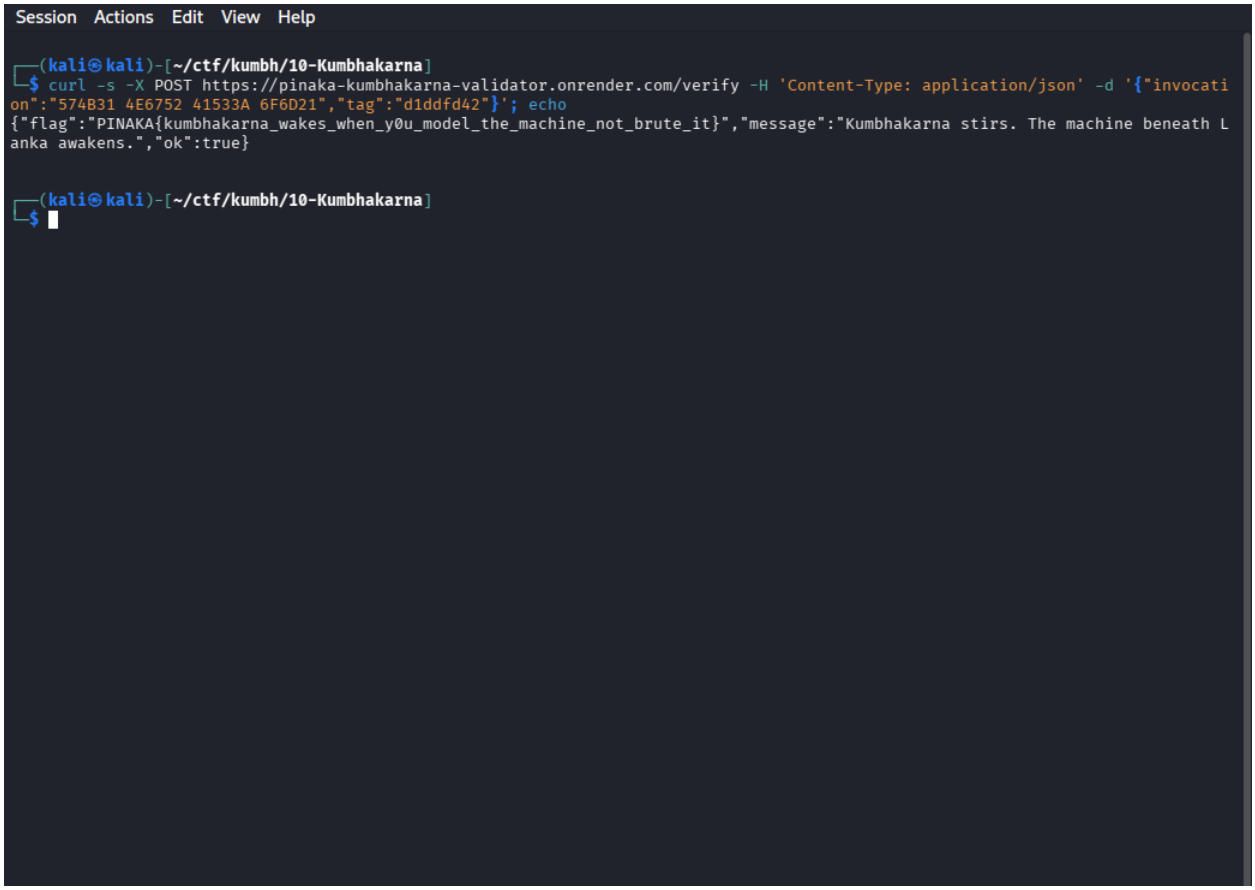


Fig 5 - validator returns the flag

Flag

```
PINAKA{kumbhakarna_wakes_when_y0u_model_the_machine_not_brute_it}
```